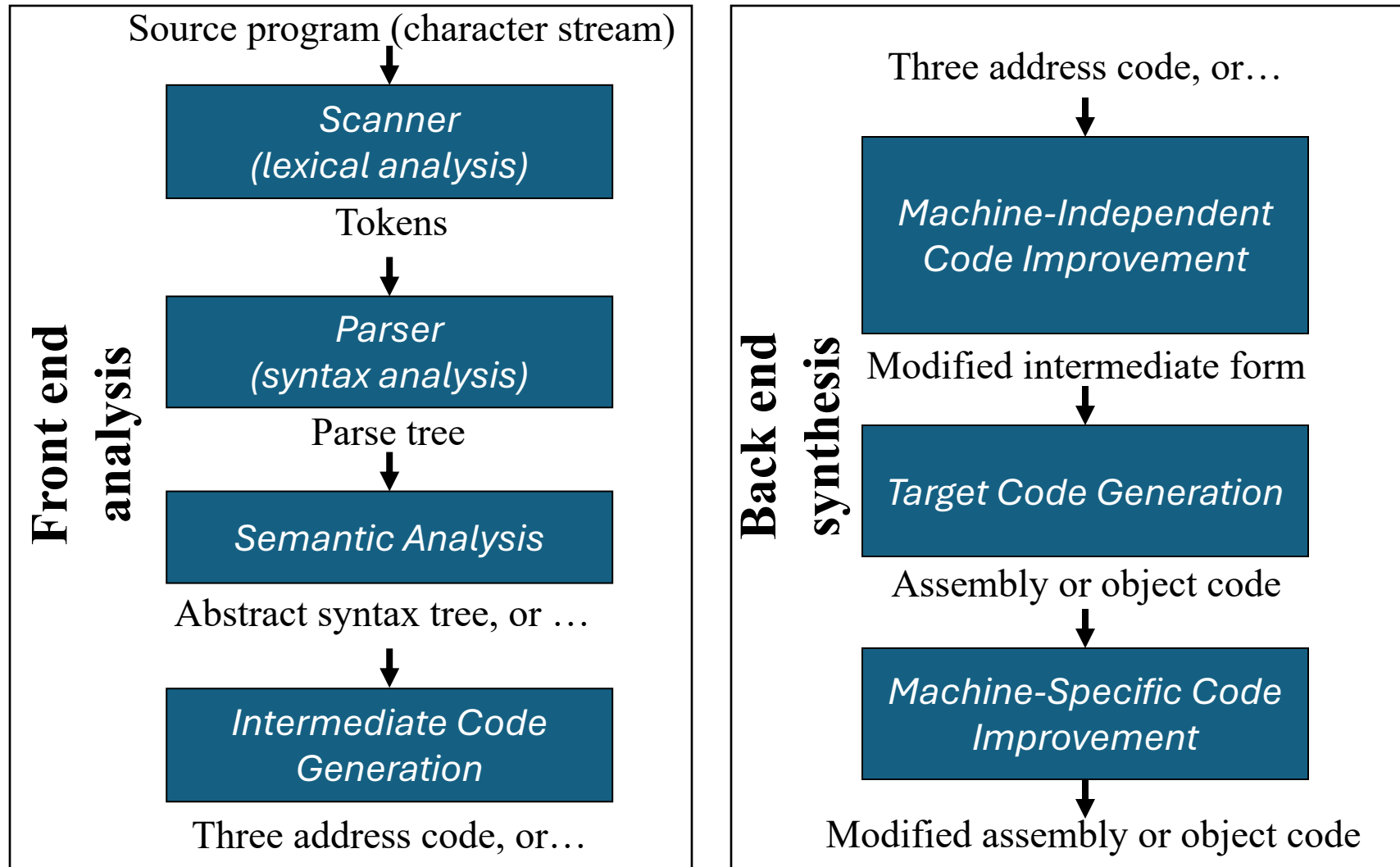


# Advanced Programming

September 25, 2025

Notes on top-down Parsing, and something else

# Compiler Front- and Back-end



# Definition of Programming Languages

A PL is defined via **syntax**, **semantics** and **pragmatics**

- The **syntax** is concerned with the form of programs: how *expressions, commands, declarations*, and other constructs must be arranged to make a well-formed program.
- The **semantics** is concerned with the meaning of (well-formed) programs: how a program may be expected to behave when executed on a computer.
- The **pragmatics** is concerned with the way in which the PL is intended to be used in practice.

# Syntax Definition: Grammars

- A **grammar** is a 4-tuple  $G = (N, T, P, S)$  where
  - $T$  is a finite set of tokens (*terminal* symbols)
  - $N$  is a finite set of *nonterminals*
  - $P$  is a finite set of *productions* of the form
$$\alpha \rightarrow \beta$$
where  $\alpha \in (N \cup T)^* N (N \cup T)^*$  and  $\beta \in (N \cup T)^*$
  - $S \in N$  is a designated *start symbol*
- $\mathbf{A}^*$  is the set of finite sequences of elements of  $\mathbf{A}$ . If  $\mathbf{A} = \{a, b\}$ ,  $\mathbf{A}^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$
- $\mathbf{AB} = \{ab \mid a \in \mathbf{A}, b \in \mathbf{B}\}$

# Notational Conventions Used

- Terminals

$a, b, c, \dots \in T$

specific terminals: **0**, **1**, **id**, **+**

- Nonterminals

$A, B, C, \dots \in N$

specific nonterminals: *expr*, *term*, *stmt*

- Grammar symbols

$X, Y, Z \in (N \cup T)$

- Strings of terminals

$u, v, w, x, y, z \in T^*$

- Strings of grammar symbols

$\alpha, \beta, \gamma \in (N \cup T)^*$

# Chomsky Hierarchy: Grammar Classification

A grammar  $G$  is said to be

- **Regular** if it is **right linear** where each production is of the form

$$A \rightarrow w B \quad \text{or} \quad A \rightarrow w$$

or **left linear** where each production is of the form

$$A \rightarrow B w \quad \text{or} \quad A \rightarrow w \quad (w \in T^*)$$

- **Context free** if each production is of the form

$$A \rightarrow \alpha$$

where  $A \in N$  and  $\alpha \in (N \cup T)^*$

- **Context sensitive** if each production is of the form

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

where  $A \in N$ ,  $\alpha, \gamma, \beta \in (N \cup T)^*$ ,  $|\gamma| > 0$

- **Unrestricted**

# Chomsky Hierarchy

$$\mathcal{L}(\text{regular}) \subset \mathcal{L}(\text{context free}) \subset \mathcal{L}(\text{context sensitive}) \subset \mathcal{L}(\text{unrestricted})$$

Where  $\mathcal{L}(T) = \{ L(G) \mid G \text{ is of type } T \}$

That is: the set of all languages  
generated by grammars  $G$  of type  $T$

Examples:

Every *finite language* is regular! (construct a FSA for strings in  $L(G)$ )

$L_1 = \{ \mathbf{a}^n \mathbf{b}^n \mid n \geq 0 \}$  is context free

$L_2 = \{ \mathbf{a}^n \mathbf{b}^n \mathbf{c}^n \mid n \geq 0 \}$  is context sensitive

# La gerarchia di Chomsky

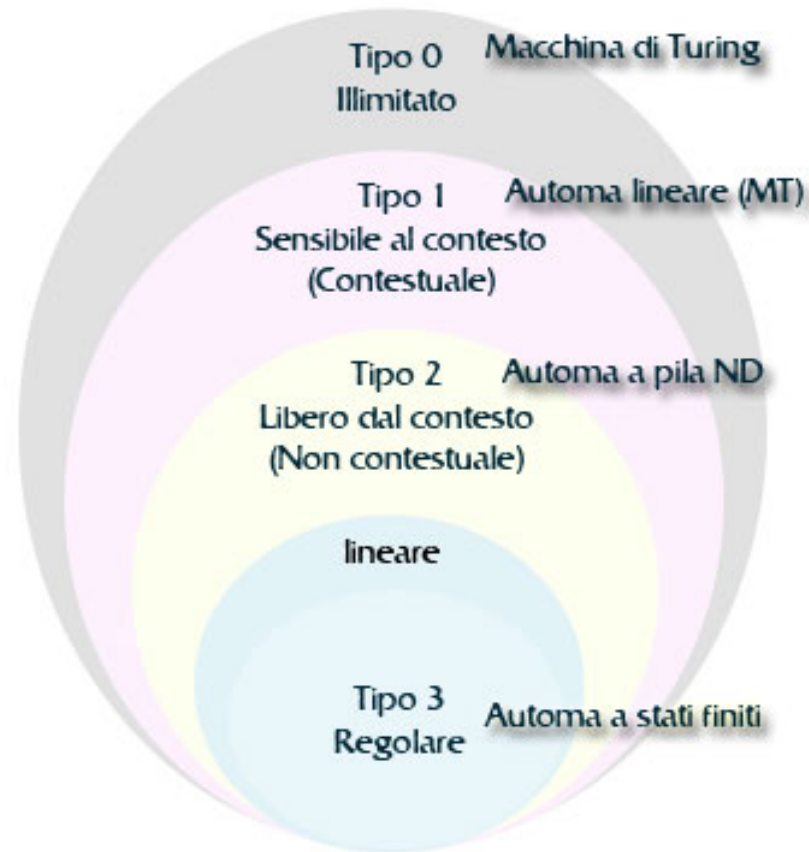


Figura 8.3: La Gerarchia di Chomsky.



# Derivations

- A *one-step derivation* is defined by

$$\gamma \alpha \delta \Rightarrow \gamma \beta \delta$$

where  $\alpha \rightarrow \beta$  is a production in the grammar

- In addition, we define
  - $\Rightarrow$  is *leftmost*  $\Rightarrow_{lm}$  if  $\gamma$  does not contain a nonterminal
  - $\Rightarrow$  is *rightmost*  $\Rightarrow_{rm}$  if  $\delta$  does not contain a nonterminal
  - Transitive closure  $\Rightarrow^*$  (zero or more steps)
  - Positive closure  $\Rightarrow^+$  (one or more steps)
- $\alpha$  is a *sentential form* if  $S \Rightarrow^* \alpha$
- The *language generated by  $G$*  is defined by

$$L(G) = \{w \in T^* \mid S \Rightarrow^+ w\}$$

# Derivation (Example)

Grammar  $G = (\{E\}, \{+, *, (, ), -, \text{id}\}, P, E)$  with productions

$$\begin{aligned} P = \quad & E \rightarrow E + E \\ & E \rightarrow E * E \\ & E \rightarrow ( E ) \\ & E \rightarrow - E \\ & E \rightarrow \text{id} \end{aligned}$$

Example derivations:

$$E \Rightarrow - E. \Rightarrow - \text{id}$$

$$E \Rightarrow_{rm} E + E \Rightarrow_{rm} E + \text{id} \Rightarrow_{rm} \text{id} + \text{id}$$

$$E \Rightarrow^* E$$

$$E \Rightarrow^* \text{id} + \text{id}$$

$$E \Rightarrow^+ \text{id} * \text{id} + \text{id}$$

# Another grammar for expressions

$$G = \langle \{list, digit\}, \{+, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, P, list \rangle$$

Productions  $P =$

$$\begin{aligned} list &\rightarrow list + digit \\ list &\rightarrow list - digit \\ list &\rightarrow digit \\ digit &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

A *leftmost* derivation:

$$\begin{aligned} &\underline{list} \\ \Rightarrow_{lm} &\underline{list} + digit \\ \Rightarrow_{lm} &\underline{list} - digit + digit \\ \Rightarrow_{lm} &\underline{digit} - digit + digit \\ \Rightarrow_{lm} &9 - \underline{digit} + digit \\ \Rightarrow_{lm} &9 - 5 + \underline{digit} \\ \Rightarrow_{lm} &9 - 5 + 2 \end{aligned}$$

# Parse Trees (context-free grammars)

- Tree-shaped representation of derivations
- The *root* of the tree is labeled by the start symbol
- Each *leaf* of the tree is labeled by a terminal (= token) or  $\epsilon$
- Each *internal node* is labeled by a nonterminal
- If an internal node is labeled by  $A$ , there is a production

$$A \rightarrow X_1 X_2 \dots X_n$$

such that the node has immediate children labeled by

$$X_1, X_2, \dots, X_n$$

or a single node labeled by  $\epsilon$  if  $A \rightarrow \epsilon$ )

# A parse tree for grammar G

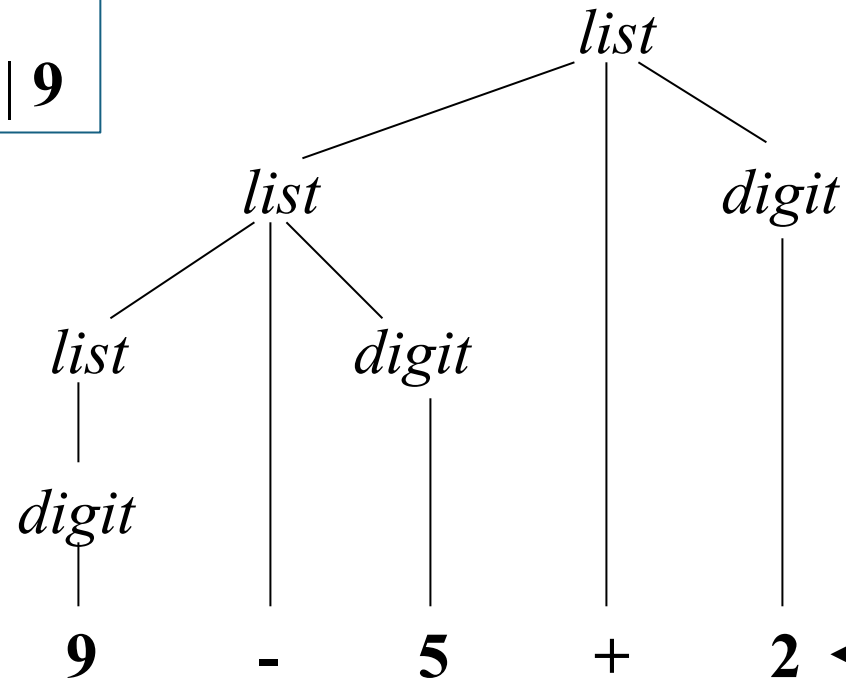
$list \rightarrow list + digit$

$list \rightarrow list - digit$

$list \rightarrow digit$

$digit \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Parse tree of the string **9-5+2**



The sequence of  
leafs is called the  
*yield* of the parse tree

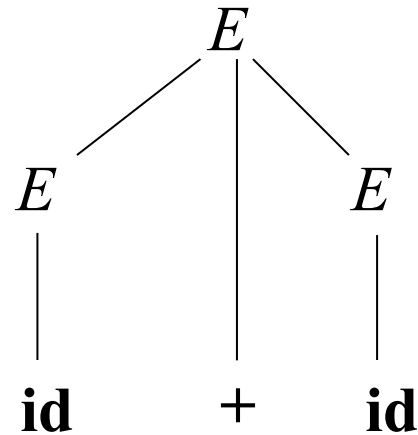
# Parse trees and derivations

- A parse tree may correspond to several derivations
- A parse tree has a unique *rightmost* (*leftmost*) derivation

$$E \Rightarrow_{rm} E + E \Rightarrow_{rm} E + \mathbf{id} \Rightarrow_{rm} \mathbf{id} + \mathbf{id}$$

$$E \Rightarrow_{lm} E + E \Rightarrow_{lm} \mathbf{id} + E \Rightarrow_{lm} \mathbf{id} + \mathbf{id}$$

$$P = E \rightarrow E + E \mid \mathbf{id}$$



# Ambiguity

Consider the following context-free grammar:

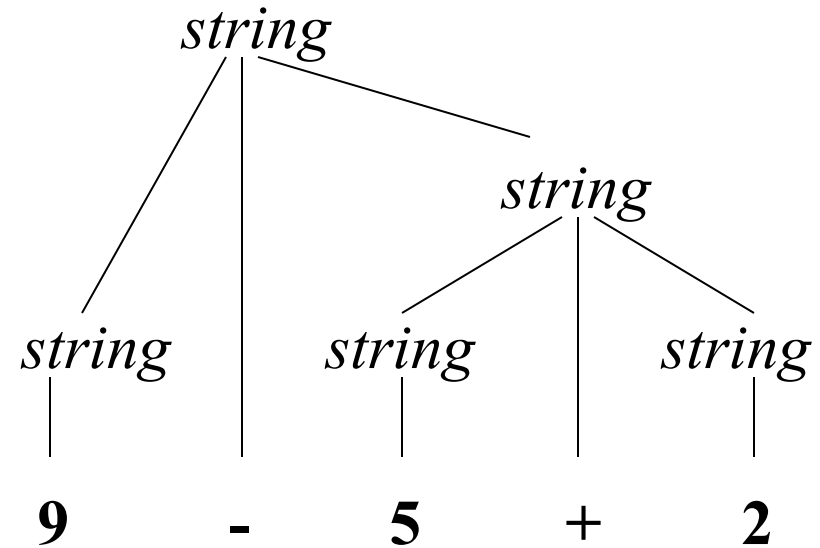
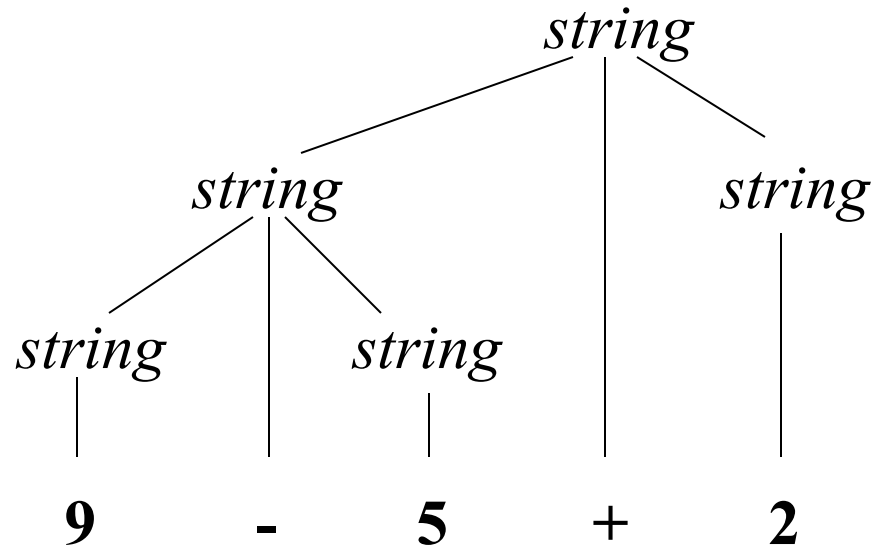
$$G = \langle \{string\}, \{+, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}, P, string \rangle$$

with production  $P =$

$$string \rightarrow string + string \mid string - string \mid 0 \mid 1 \mid \dots \mid 9$$

This grammar is *ambiguous*, because more than one parse tree represents the string **9-5+2**

# Ambiguity (cont'd)





# The syntax of programming languages

- The syntax of a programming language is typically defined by two grammars
  - **Lexical grammar**
    - **Regular**, often presented as regular expressions
    - Terminal symbols are characters
    - Defines tokens
  - **Syntax grammar**
    - **Context-free**, often presented in Backus-Naur form
    - Terminal symbols are tokens
    - Defines constructs of the language, not expressible with REs
- Note: there are **non-context free** syntact constructs
  - Variables are declared before use  $\rightarrow \{wcw \mid w \in (a \mid b)^*\}$
  - Number of actual/formal parameters  $\rightarrow \{a^n b^m c^n d^m \mid n > 0, m > 0\}$

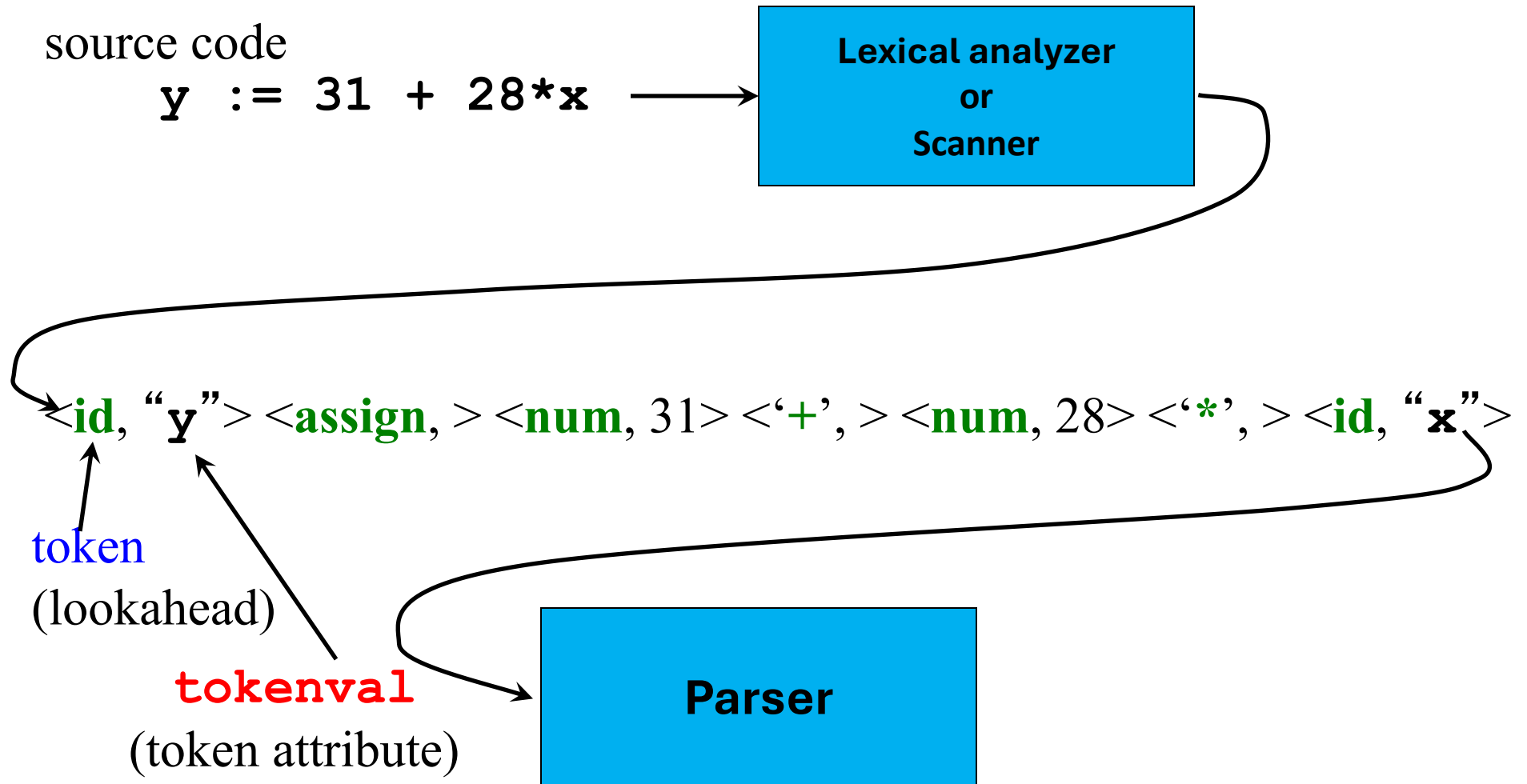
# Where can I find the syntax of a PL?

- Java
  - <https://docs.oracle.com/javase/specs/index.html>
  - Chapter 19 of Java Language Specification
- ...

# The Reason Why Lexical Analysis is a Separate Phase

- Simplifies the design of the compiler
  - LL(1) or LR(1) parsing with 1 token lookahead would not be possible (multiple characters/tokens to match)
- Provides efficient implementation
  - Systematic techniques to implement lexical analyzers by hand or automatically from specifications
  - Stream buffering methods to scan input
- Improves portability
  - Non-standard symbols and alternate character encodings can be normalized (e.g. UTF8, trigraphs)

# Main goal of lexical analysis: tokenization



# Additional tasks of the Lexical Analyzer

- Remove comments and useless white spaces / tabs from the source code
- Correlate error messages of the parser with source code (e.g. keeping track of line numbers)
- Expansion of macros

# Parsing

- Parsing = *process of determining if a string of tokens can be generated by a grammar*
- For any CF grammar there is a parser that takes at most  $O(n^3)$  time to parse a string of  $n$  tokens
- **Linear** algorithms suffice for parsing programming language source code
- *Top-down parsing* “constructs” a parse tree from root to leaves
- *Bottom-up parsing* “constructs” a parse tree from leaves to root

# Parsing algorithms

- *Universal* (any C-F grammar)
  - Cocke-Younger-Kasimi, Earley
  - Based on dynamic programming,  $O(n^3)$
- *Top-down* (C-F grammar with restrictions)
  - Recursive descent (predictive parsing)
  - LL (Left-to-right, Leftmost derivation) methods
  - Linear on certain grammars; easier to do manually
- *Bottom-up* (C-F grammar with restrictions)
  - Operator precedence parsing
  - LR (Left-to-right, Rightmost derivation) methods
    - SLR, canonical LR, LALR
  - Linear on certain grammars; typically generated by tools

# Top-down parsing: Recursive descent parsing

- ***Recursive descent parsing*** is a top-down parsing method
  - Each nonterminal has one (recursive) procedure that is responsible for parsing the nonterminal's syntactic category of input tokens
  - When a nonterminal has multiple productions, they are tried in sequence till one succeeds. If all fail, the procedure for the non-terminal fails.
  - Backtracking is necessary, and complexity is exponential in general



# Generic procedure for a non-terminal $A$ in a recursive descent parsing

```
void A() {  
    Choose an  $A$ -production,  $A \rightarrow X_1 X_2 \cdots X_k$ ;  
    for (  $i = 1$  to  $k$  ) {  
        if (  $X_i$  is a nonterminal )  
            call procedure  $X_i()$ ;  
        else if (  $X_i$  equals the current input symbol  $a$  )  
            advance the input to the next symbol;  
        else /* an error has occurred */;  
    }  
}
```

# Recursive descent parsing for

```
type → simple
      | ^ id
      | array [ simple ] of type
simple → integer
      | char
      | num dotdot num
```

```
# input: array di token
i ← 0

function parse(input):
  i ← 0
  if TYPE() and i = len(input) then return ok else return fail

# --- Non-terminal i ---

function TYPE(): # type → simple | '^' id | array '[' simple ']' of type
  mark ← i
  # 1) '^' id
  if match('^') and match('id'):
    return ok
  i ← mark

  # 2) array '[' simple ']' of type
  if match('array') and match('[') and SIMPLE() and match(']') and
  match('of') and TYPE():
    return ok
  i ← mark
```

```
# 3) simple
if SIMPLE():
  return ok
return fail

function SIMPLE(): # simple → integer | char | num dotdot num
  mark ← i
  if match('integer'): return ok
  i ← mark
  if match('char'): return ok
  i ← mark
  if match('num') and match('dotdot') and match('num'):
    return ok
  return fail

# --- Primitive ---

function match(t):
  if i < len(input) and input[i] = t:
    i ← i + 1
    return true
  return false
```

# Top-down parsing: Predictive parsing

- ***Predictive parsing*** is a special form of recursive descent parsing where one uses **lookahead tokens** to unambiguously determine the production to try. Complexity is linear.
- Critical point: to determine which lookahead tokens trigger which production
- Obtained using the **FIRST** and **FOLLOW** sets of tokens for each non-terminal

# Example Predictive Parser

```
type → simple
      | ^ id
      | array [ simple ] of type
simple → integer
      | char
      | num dotdot num
```

```
procedure type();
begin
  if lookahead in { 'integer', 'char', 'num' }
  then
    simple()
  else if lookahead = '^' then
    match('^'); match(id)
  else if lookahead = 'array' then
    match('array'); match('['); simple();
    match(']'); match('of'); type()
  else error()
end;
```

```
procedure match(t : token);
begin
  if lookahead = t then
    lookahead := nexttoken()
  else error()
end;
```

```
procedure simple();
begin
  if lookahead = 'integer' then
    match('integer')
  else if lookahead = 'char' then
    match('char')
  else if lookahead = 'num' then
    match('num');
    match('dotdot');
    match('num')
  else error()
end;
```



# Example Predictive Parser (Execution Step 2)

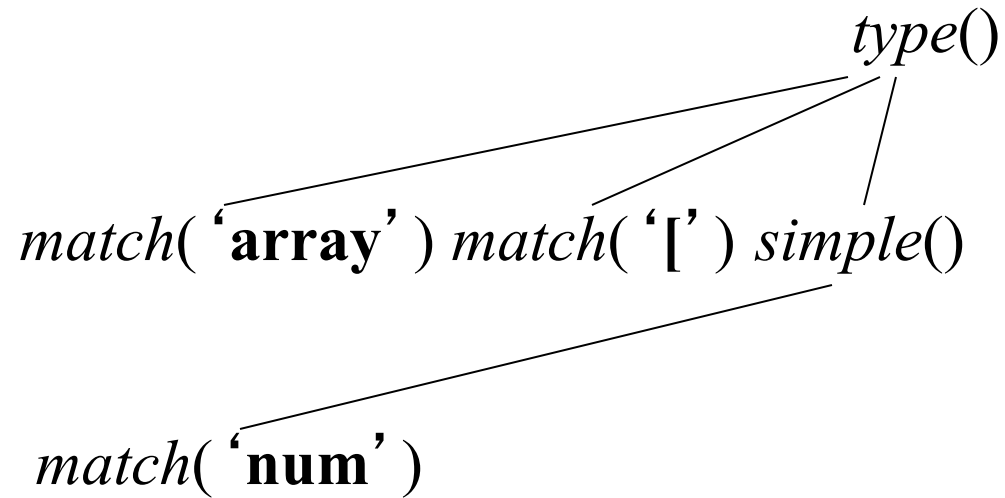
*match()*

*match( 'array' ) match( '[' )*

Input:      array    [       num     dotdot     num    ]    of   integer

                ↑  
              *lookahead*

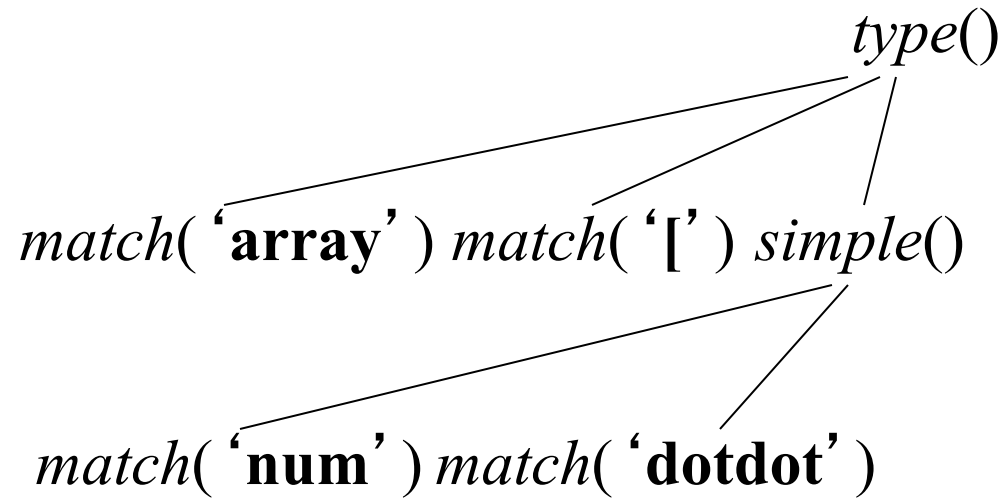
# Example Predictive Parser (Execution Step 3)



Input:      array    [     num       dotdot       num    ]    of   integer

                                ↑  
lookahead

# Example Predictive Parser (Execution Step 4)

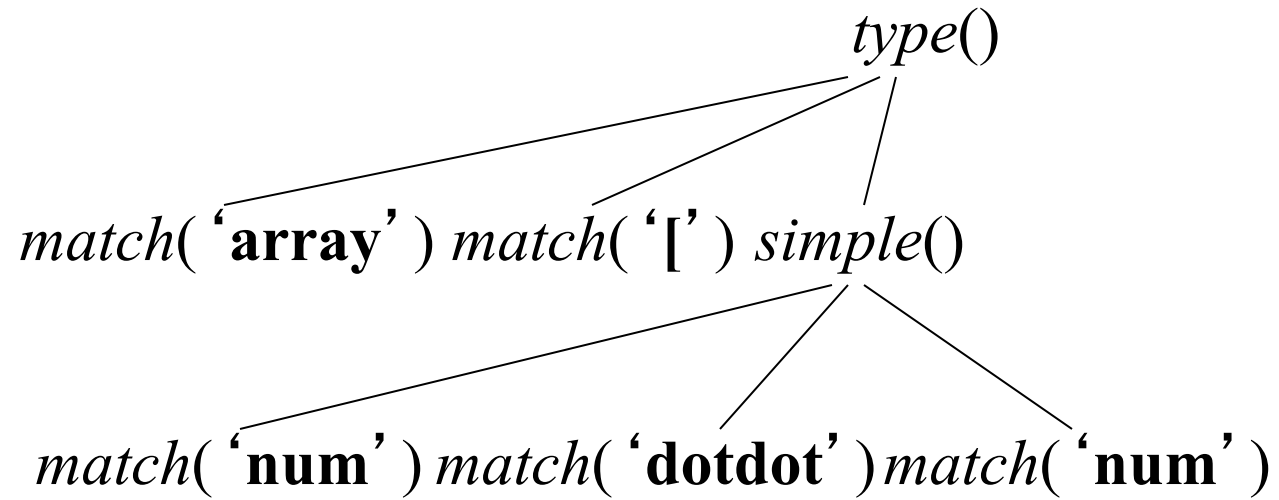


Input:      **array**    [    num    **dotdot**    num    ]    **of integer**

                                ↑  
*lookahead*

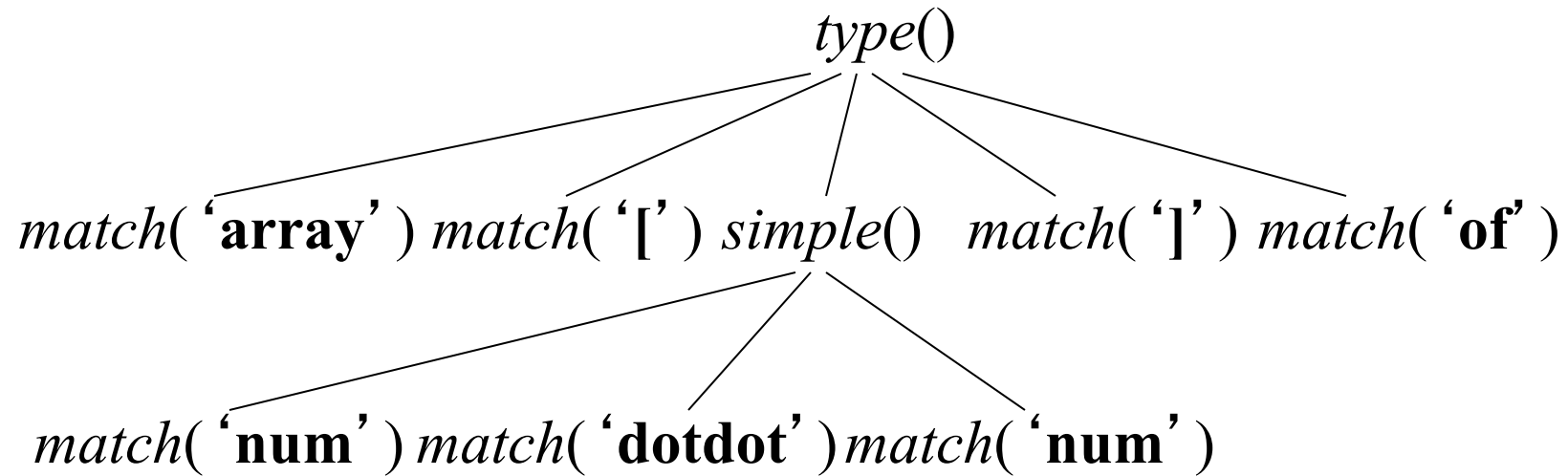


# Example Predictive Parser (Execution Step 5)

[illegible]



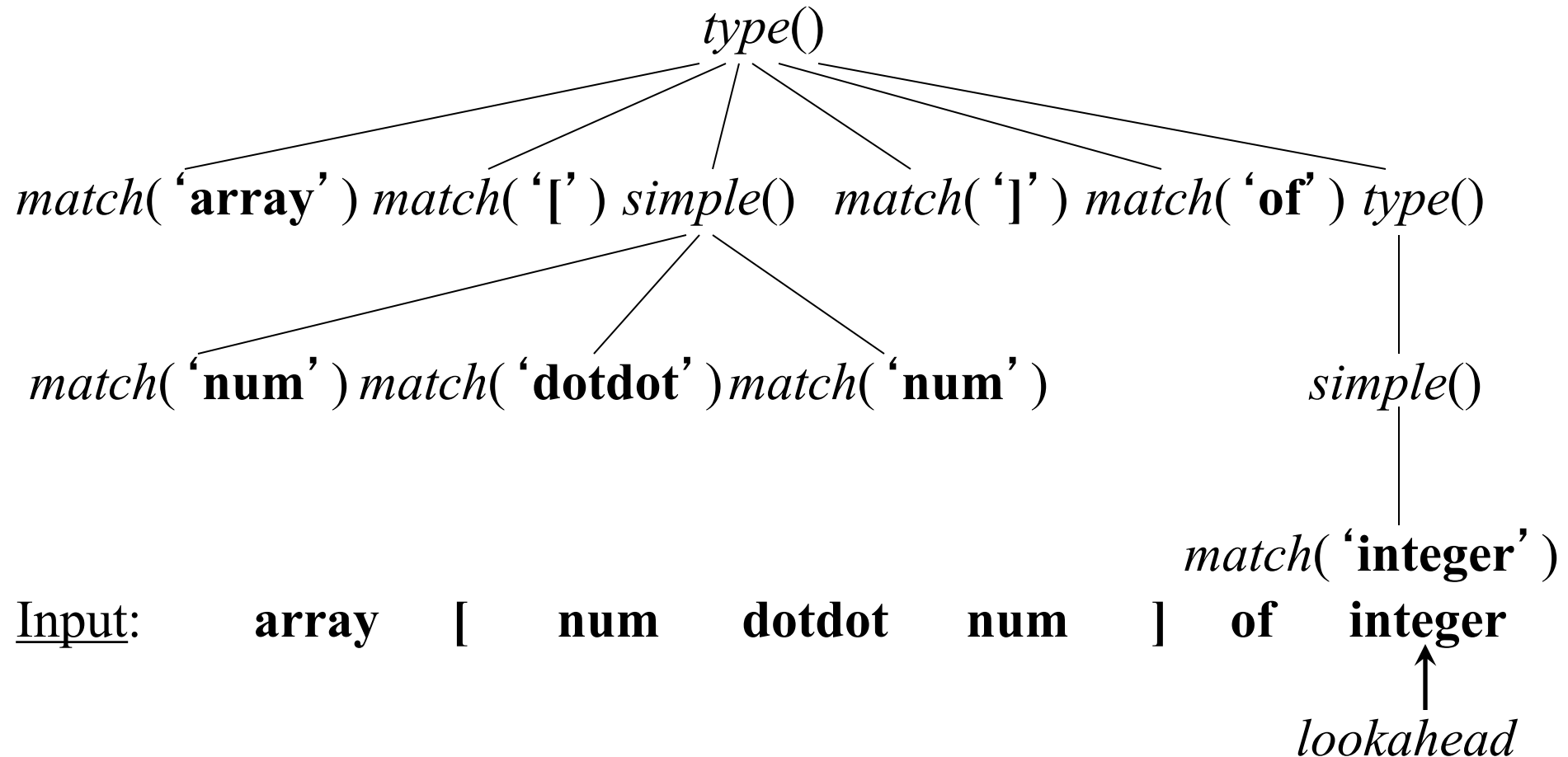
# Example Predictive Parser (Execution Step 7)



Input:      array    [    num    dotdot    num    ]    of    integer

↑  
*lookahead*

# Example Predictive Parser (Execution Step 8)



# FIRST

$\text{FIRST}(\alpha)$  is the set of terminals that appear as the first symbols of one or more strings generated from  $\alpha$

$$\begin{aligned} \text{type} &\rightarrow \text{simple} \\ &\quad | \text{ } ^\wedge \text{ id} \\ &\quad | \text{ array [ simple ] of type} \\ \text{simple} &\rightarrow \text{integer} \\ &\quad | \text{ char} \\ &\quad | \text{ num dotdot num} \end{aligned}$$

$\text{FIRST}(\text{simple}) = \{ \text{integer, char, num} \}$

$\text{FIRST}(^\wedge \text{ id}) = \{ ^\wedge \}$

$\text{FIRST}(\text{type}) = \{ \text{integer, char, num, } ^\wedge, \text{ array} \}$

# How to use FIRST

We use FIRST to write a predictive parser as follows

|                                  |  |                                                                                 |
|----------------------------------|--|---------------------------------------------------------------------------------|
| $expr \rightarrow term\ rest$    |  | <b>procedure</b> <i>rest</i> ();                                                |
| $rest \rightarrow +\ term\ rest$ |  | <b>begin</b>                                                                    |
| $- \ term\ rest$                 |  | <b>if</b> <i>lookahead</i> in <u>FIRST(+ <i>term rest</i>)</u> <b>then</b>      |
| $\epsilon$                       |  | <i>match</i> ( '+' ); <i>term</i> (); <i>rest</i> ()                            |
|                                  |  | <b>else if</b> <i>lookahead</i> in <u>FIRST(- <i>term rest</i>)</u> <b>then</b> |
|                                  |  | <i>match</i> ( '-' ); <i>term</i> (); <i>rest</i> ()                            |
|                                  |  | <b>else return</b>                                                              |
|                                  |  | <b>end;</b>                                                                     |

When a nonterminal  $A$  has two (or more) productions as in

$$\begin{array}{l} A \rightarrow \alpha \\ \quad | \beta \end{array}$$

then **FIRST**( $\alpha$ ) and **FIRST**( $\beta$ ) must be disjoint for predictive parsing to work

# Left Factoring

- If a nonterminal has two or more productions whose right-hand sides start with the same symbol, the grammar is not LL(1)

- Example:

- $stmt ::= \text{if } expr \text{ then } stmt \text{ else } stmt$   
|  $\text{if } expr \text{ then } stmt$

- Solution: replace productions

$$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \dots \mid \alpha \beta_n \mid \gamma$$

with

$$A \rightarrow \alpha A_R \mid \gamma$$

$$A_R \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

- Example:

- $stmt ::= \text{if } expr \text{ then } stmt \text{ stmt}'$
  - $stmt' ::= \text{else } stmt \mid \epsilon$

# Left Recursion

- A grammar is *left-recursive* if there is a non-terminal  $A$  such that  $A \Rightarrow^+ A\eta$  for some string  $\eta$ 
  - Example of immediate left-recursion:
$$A \rightarrow A\alpha \mid A\beta \mid \gamma \mid \delta$$
  - Left recursion can be indirect
- If the grammar is left-recursive, it cannot be  $LL(k)$ : a top-down parser loops forever on certain inputs
- Immediate left recursion elimination:

$$A \rightarrow \gamma A_R \mid \delta A_R$$

$$A_R \rightarrow \alpha A_R \mid \beta A_R \mid \varepsilon$$



# A General Left Recursion Elimination Method

- *Input: Grammar  $G$  with no cycles or  $\varepsilon$ -productions*
- Arrange the nonterminals in some order  $A_1, A_2, \dots, A_n$

**for**  $i = 1, \dots, n$  **do**

**for**  $j = 1, \dots, i-1$  **do**

        replace each

$$A_i \rightarrow A_j \gamma$$

    with

$$A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$$

    where

$$A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$$

**enddo**

    eliminate the *immediate left recursion* in  $A_i$

**enddo**

# Example of left-recursion elimination: Grammars for expressions

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow ( E ) \mid \mathbf{id} \end{aligned}$$

Grammar after left recursion elimination

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \epsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \epsilon \\ F &\rightarrow ( E ) \mid \mathbf{id} \end{aligned}$$

# Predictive Parsing

- Eliminate left recursion from grammar
- Left factor the grammar
- Compute FIRST and FOLLOW, and check if the grammar is LL(1)
- FIRST and FOLLOW are used in the parsing algorithm
- Two variants:
  - Recursive (recursive-descent parsing)
  - Non-recursive (table-driven parsing)

# FIRST (Revisited)

- $\text{FIRST}(\alpha) = \{ \quad \quad \quad \text{the set of terminals that begin all strings derived from } \alpha \}$
- $\text{FIRST}(a) = \{a\} \quad \quad \quad \text{if } a \in T$
- $\text{FIRST}(\varepsilon) = \{\varepsilon\}$
- $\text{FIRST}(A) = \cup_{A \rightarrow \alpha} \text{FIRST}(\alpha) \quad \quad \quad \text{for } A \rightarrow \alpha \in P$
- $\text{FIRST}(X_1X_2...X_k) =$ 
  - for all**  $i = 1, \dots, k$  **do**
  - if** for all  $j = 1, \dots, i-1 : \varepsilon \in \text{FIRST}(X_j)$  **then**
    - add non- $\varepsilon$  in  $\text{FIRST}(X_i)$  to  $\text{FIRST}(X_1X_2...X_k)$
  - if** for all  $j = 1, \dots, k : \varepsilon \in \text{FIRST}(X_j)$  **then**
    - add  $\varepsilon$  to  $\text{FIRST}(X_1X_2...X_k)$

# FOLLOW

- $\text{FOLLOW}(A) = \{ \text{the set of terminals that can immediately follow nonterminal } A \}$
- $\text{FOLLOW}(A) =$ 
  - for** all  $(B \rightarrow \alpha A \beta) \in P$  **do**
    - add  $\text{FIRST}(\beta) \setminus \{\epsilon\}$  to  $\text{FOLLOW}(A)$
  - for** all  $(B \rightarrow \alpha A \beta) \in P$  and  $\epsilon \in \text{FIRST}(\beta)$  **do**
    - add  $\text{FOLLOW}(B)$  to  $\text{FOLLOW}(A)$
  - for** all  $(B \rightarrow \alpha A) \in P$  **do**
    - add  $\text{FOLLOW}(B)$  to  $\text{FOLLOW}(A)$
  - if**  $A$  is the start symbol  $S$  **then**
    - add  $\$$  to  $\text{FOLLOW}(A)$

# LL(1) Grammar

- A grammar  $G$  is LL(1) if it is not left recursive and for each collection of productions

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$$

for nonterminal  $A$  the following holds:

1.  $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \emptyset$  for all  $i \neq j$
2. if  $\alpha_i \Rightarrow^* \varepsilon$  then
  - 2.a.  $\alpha_j \not\Rightarrow^* \varepsilon$  for all  $i \neq j$
  - 2.b.  $\text{FIRST}(\alpha_j) \cap \text{FOLLOW}(A) = \emptyset$  for all  $i \neq j$

# Non-LL(1) Examples

| <i>Grammar</i>                                                                      | <i>Not LL(1) because:</i>                                                           |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| $S \rightarrow S \mathbf{a} \mid \mathbf{a}$                                        | Left recursive                                                                      |
| $S \rightarrow \mathbf{a} S \mid \mathbf{a}$                                        | $\text{FIRST}(\mathbf{a} S) \cap \text{FIRST}(\mathbf{a}) \neq \emptyset$           |
| $S \rightarrow \mathbf{a} R \mid \varepsilon$<br>$R \rightarrow S \mid \varepsilon$ | For $R$ : $S \Rightarrow^* \varepsilon$ and $\varepsilon \Rightarrow^* \varepsilon$ |
| $S \rightarrow \mathbf{a} R \mathbf{a}$<br>$R \rightarrow S \mid \varepsilon$       | For $R$ :<br>$\text{FIRST}(S) \cap \text{FOLLOW}(R) \neq \emptyset$                 |

# Predictive Recursive Descent Parsing

- Grammar must be LL(1)
- Every nonterminal has one (recursive) procedure responsible for parsing the nonterminal's syntactic category of input tokens
- When a nonterminal has multiple productions, each production is implemented in a branch of a selection statement based on input look-ahead information



# FIRST and FOLLOW: An Example

$$\begin{array}{lll} E & \rightarrow & T E' \\ E' & \rightarrow & + T E' \mid \epsilon \\ T & \rightarrow & F T' \\ T' & \rightarrow & * F T' \mid \epsilon \\ F & \rightarrow & ( E ) \mid \mathbf{id} \end{array}$$

- $\text{FIRST}(F) = \text{FIRST}(T) = \text{FIRST}(E) = \{ (, \mathbf{id} \}$
- $\text{FIRST}(E') = \{ +, \epsilon \}$
- $\text{FIRST}(T') = \{ *, \epsilon \}$
- $\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{ ), \$ \}$
- $\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{ +, ), \$ \}$
- $\text{FOLLOW}(F) = \{ +, *, ), \$ \}$

# Using FIRST and FOLLOW in a Recursive-Descent Parser

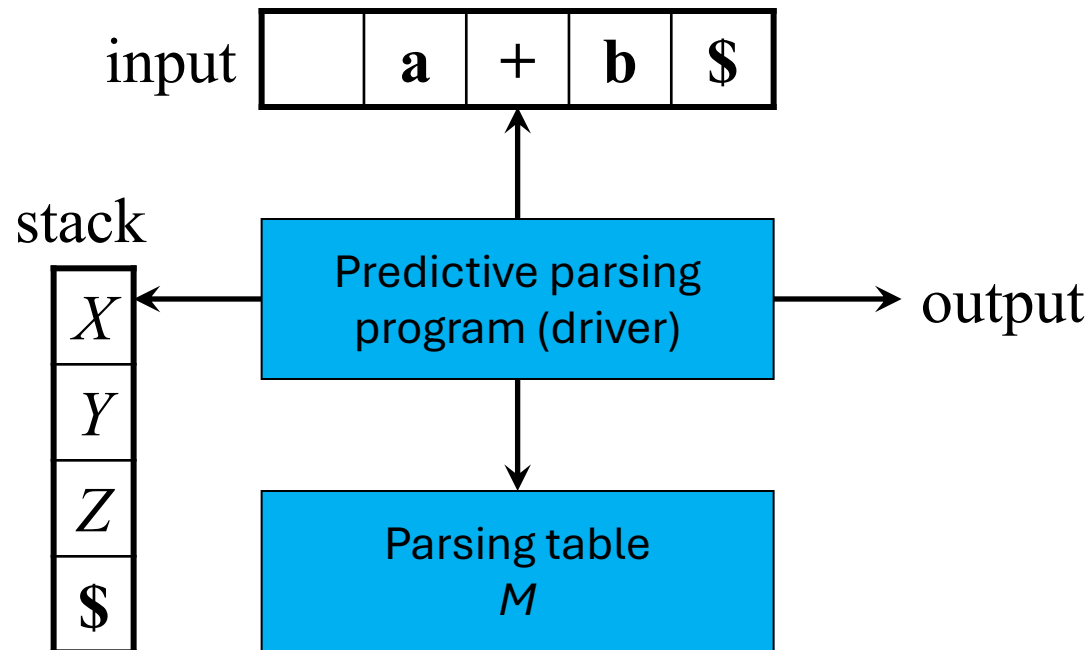
$expr \rightarrow term\ rest$   
 $rest \rightarrow +\ term\ rest$   
           $| -\ term\ rest$   
           $| \epsilon$   
 $term \rightarrow id$

```
procedure rest();  
begin  
  if lookahead in FIRST(+ term rest) then  
    match( '+' ); term(); rest()  
  else if lookahead in FIRST(- term rest) then  
    match( '-' ); term(); rest()  
  else if lookahead in FOLLOW(rest) then  
    return  
  else error()  
end;
```

where  $FIRST(+\ term\ rest) = \{ + \}$   
 $FIRST(-\ term\ rest) = \{ - \}$   
 $FOLLOW(rest) = \{ \$ \}$

# Non-Recursive Predictive Parsing: Table-Driven Parsing

- Given an LL(1) grammar  $G = (N, T, P, S)$  construct a table  $M$  and use a *driver program* with a *stack*
- The stack “replaces the runtime stack” of the recursive algorithm. It will contain symbols of the grammar.



# Constructing an LL(1) Predictive Parsing Table

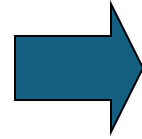
- Table  $M$  has one entry  $M[A, a]$  for each  $A \in N$  and  $a \in T$
- Entry  $M[A, a]$  contains the production to apply when  $A$  has to be reduced and  $a$  is the lookahead

```
for each production  $A \rightarrow \alpha$  do  
    for each  $a \in \text{FIRST}(\alpha)$  do  
        add production  $A \rightarrow \alpha$  to  $M[A, a]$   
    enddo  
    if  $\epsilon \in \text{FIRST}(\alpha)$  then  
        for each  $b \in \text{FOLLOW}(A)$  do  
            add  $A \rightarrow \alpha$  to  $M[A, b]$   
        enddo  
    endif  
enddo
```

- Mark each undefined entry in  $M$  error
- **Note:** The grammar is LL(1) iff  $M[A, a]$  contains at most one production for each  $A \in N$  and  $a \in T$

# Example Table

$E \rightarrow T E_R$   
 $E_R \rightarrow + T E_R \mid \varepsilon$   
 $T \rightarrow F T_R$   
 $T_R \rightarrow * F T_R \mid \varepsilon$   
 $F \rightarrow ( E ) \mid \mathbf{id}$



| $A \rightarrow \alpha$        | FIRST( $\alpha$ ) | FOLLOW( $A$ ) |
|-------------------------------|-------------------|---------------|
| $E \rightarrow T E_R$         | ( <b>id</b>       | \$ )          |
| $E_R \rightarrow + T E_R$     | +                 | \$ )          |
| $E_R \rightarrow \varepsilon$ | $\varepsilon$     |               |
| $T \rightarrow F T_R$         | ( <b>id</b>       | + \$ )        |
| $T_R \rightarrow * F T_R$     | *                 | + \$ )        |
| $T_R \rightarrow \varepsilon$ | $\varepsilon$     |               |
| $F \rightarrow ( E )$         | (                 | * + \$ )      |
| $F \rightarrow \mathbf{id}$   | <b>id</b>         | * + \$ )      |



|       | <b>id</b>                   | +                             | *                         | (                     | )                             | \$                            |
|-------|-----------------------------|-------------------------------|---------------------------|-----------------------|-------------------------------|-------------------------------|
| $E$   | $E \rightarrow T E_R$       |                               |                           | $E \rightarrow T E_R$ |                               |                               |
| $E_R$ |                             | $E_R \rightarrow + T E_R$     |                           |                       | $E_R \rightarrow \varepsilon$ | $E_R \rightarrow \varepsilon$ |
| $T$   | $T \rightarrow F T_R$       |                               |                           | $T \rightarrow F T_R$ |                               |                               |
| $T_R$ |                             | $T_R \rightarrow \varepsilon$ | $T_R \rightarrow * F T_R$ |                       | $T_R \rightarrow \varepsilon$ | $T_R \rightarrow \varepsilon$ |
| $F$   | $F \rightarrow \mathbf{id}$ |                               |                           | $F \rightarrow ( E )$ |                               |                               |

# Predictive Parsing Program (Driver)

```
push($)  
push(S)  
a := lookahead  
repeat  
     $X := \text{pop}()$   
    if  $X$  is a terminal or  $X = \$$  then  
        match(X) // moves to next token and a := lookahead  
    else if  $M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$  then  
        push( $Y_k, Y_{k-1}, \dots, Y_2, Y_1$ ) // such that  $Y_1$  is on top  
        ... invoke actions and/or produce IR output ...  
    else    error()  
    endif  
until  $X = \$$ 
```

# Example Table-Driven Parsing

| Stack                             | Input                                         | Production applied            |
|-----------------------------------|-----------------------------------------------|-------------------------------|
| $\$E$                             | $\underline{\text{id}}+\text{id}*\text{id}\$$ | $E \rightarrow T E_R$         |
| $\$E_R T$                         | $\underline{\text{id}}+\text{id}*\text{id}\$$ | $T \rightarrow F T_R$         |
| $\$E_R T_R F$                     | $\underline{\text{id}}+\text{id}*\text{id}\$$ | $F \rightarrow \text{id}$     |
| $\$E_R T_R \underline{\text{id}}$ | $\underline{\text{id}}+\text{id}*\text{id}\$$ |                               |
| $\$E_R T_R$                       | $\underline{+}\text{id}*\text{id}\$$          | $T_R \rightarrow \varepsilon$ |
| $\$E_R$                           | $\underline{+}\text{id}*\text{id}\$$          | $E_R \rightarrow + T E_R$     |
| $\$E_R T \underline{+}$           | $\underline{+}\text{id}*\text{id}\$$          |                               |
| $\$E_R T$                         | $\underline{\text{id}}*\text{id}\$$           | $T \rightarrow F T_R$         |
| $\$E_R T_R F$                     | $\underline{\text{id}}*\text{id}\$$           | $F \rightarrow \text{id}$     |

| Stack                             | Input                               | Prod. applied                 |
|-----------------------------------|-------------------------------------|-------------------------------|
| $\$E_R T_R \underline{\text{id}}$ | $\underline{\text{id}}*\text{id}\$$ |                               |
| $\$E_R T_R$                       | $\underline{*}\text{id}\$$          | $T_R \rightarrow * F T_R$     |
| $\$E_R T_R F \underline{*}$       | $\underline{*}\text{id}\$$          |                               |
| $\$E_R T_R F$                     | $\underline{\text{id}}\$$           | $F \rightarrow \text{id}$     |
| $\$E_R T_R \underline{\text{id}}$ | $\underline{\text{id}}\$$           |                               |
| $\$E_R T_R$                       | $\underline{\$}$                    | $T_R \rightarrow \varepsilon$ |
| $\$E_R$                           | $\underline{\$}$                    | $E_R \rightarrow \varepsilon$ |
| $\underline{\$}$                  | $\underline{\$}$                    |                               |

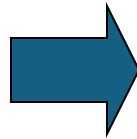
|       | id                        | +                             | *                         | (                     | )                             | \$                            |
|-------|---------------------------|-------------------------------|---------------------------|-----------------------|-------------------------------|-------------------------------|
| $E$   | $E \rightarrow T E_R$     |                               |                           | $E \rightarrow T E_R$ |                               |                               |
| $E_R$ |                           | $E_R \rightarrow + T E_R$     |                           |                       | $E_R \rightarrow \varepsilon$ | $E_R \rightarrow \varepsilon$ |
| $T$   | $T \rightarrow F T_R$     |                               |                           | $T \rightarrow F T_R$ |                               |                               |
| $T_R$ |                           | $T_R \rightarrow \varepsilon$ | $T_R \rightarrow * F T_R$ |                       | $T_R \rightarrow \varepsilon$ | $T_R \rightarrow \varepsilon$ |
| $F$   | $F \rightarrow \text{id}$ |                               |                           | $F \rightarrow ( E )$ |                               |                               |

# LL(1) Grammars are Unambiguous

Ambiguous grammar

$$S \rightarrow \mathbf{i} E \mathbf{t} S S_R \mid \mathbf{a}$$

$$S_R \rightarrow \mathbf{e} S \mid \varepsilon$$

$$E \rightarrow \mathbf{b}$$


| $A \rightarrow \alpha$                        | FIRST( $\alpha$ ) | FOLLOW( $A$ ) |
|-----------------------------------------------|-------------------|---------------|
| $S \rightarrow \mathbf{i} E \mathbf{t} S S_R$ | <b>i</b>          | <b>e \$</b>   |
| $S \rightarrow \mathbf{a}$                    | <b>a</b>          |               |
| $S_R \rightarrow \mathbf{e} S$                | <b>e</b>          | <b>e \$</b>   |
| $S_R \rightarrow \varepsilon$                 | $\varepsilon$     |               |
| $E \rightarrow \mathbf{b}$                    | <b>b</b>          | <b>t</b>      |

Error: duplicate table entry

|       | <b>a</b>                   | <b>b</b>                   | <b>e</b>                                                        | <b>i</b>                                      | <b>t</b> | <b>\$</b>                     |
|-------|----------------------------|----------------------------|-----------------------------------------------------------------|-----------------------------------------------|----------|-------------------------------|
| $S$   | $S \rightarrow \mathbf{a}$ |                            |                                                                 | $S \rightarrow \mathbf{i} E \mathbf{t} S S_R$ |          |                               |
| $S_R$ |                            |                            | $S_R \rightarrow \varepsilon$<br>$S_R \rightarrow \mathbf{e} S$ |                                               |          | $S_R \rightarrow \varepsilon$ |
| $E$   |                            | $E \rightarrow \mathbf{b}$ |                                                                 |                                               |          |                               |



# Error Handling

- A good compiler should assist in identifying and locating errors
  - *Lexical errors*: compiler can easily recover and continue (e.g. misspelled identifiers)
  - *Syntax errors*: can almost always recover (e.g. missing ';' or '{', misplaced **case**)
  - *Static semantic errors*: can sometimes recover (e.g. type mismatches, variable used before declaration)
  - *Dynamic semantic errors*: hard or impossible to detect at compile time, runtime checks are required (e.g. null pointer, division by zero, invalid array access)
  - *Logical errors*: hard or impossible to detect (e.g. `if (b = true) ...` )

# Viable-Prefix Property

- The *viable-prefix property* of parsers allows early detection of syntax errors
  - Enjoyed by LL(1), LR(1) parsers
  - Goal: detection of an error *as soon as possible* without further consuming unnecessary input
  - How: detect an error as soon as the prefix of the input does not match a prefix of any string in the language

Prefix { ...  
          **for** ( ; )  
          ...  
                    ↓ Error is  
                    detected here

# Error Recovery Strategies

- *Panic mode*
  - Discard input until a token in a set of designated “synchronizing tokens” is found (e.g. “}”, “;”)
- *Phrase-level recovery*
  - Perform local correction on the input to repair the error
- *Error productions*
  - Augment grammar with productions for erroneous constructs
- *Global correction*
  - Choose a minimal sequence of changes to obtain a global least-cost correction

# Panic Mode Recovery

Add synchronizing actions to  
undefined entries based on FOLLOW

Pro: Can be automated

Cons: Error messages are needed

$\text{FOLLOW}(E) = \{ ) \$ \}$

$\text{FOLLOW}(E_R) = \{ ) \$ \}$

$\text{FOLLOW}(T) = \{ + ) \$ \}$

$\text{FOLLOW}(T_R) = \{ + ) \$ \}$

$\text{FOLLOW}(F) = \{ + * ) \$ \}$

|       | id                        | +                          | *                         | (                     | )                          | \$                         |
|-------|---------------------------|----------------------------|---------------------------|-----------------------|----------------------------|----------------------------|
| $E$   | $E \rightarrow T E_R$     |                            |                           | $E \rightarrow T E_R$ | <i>synch</i>               | <i>synch</i>               |
| $E_R$ |                           | $E_R \rightarrow + T E_R$  |                           |                       | $E_R \rightarrow \epsilon$ | $E_R \rightarrow \epsilon$ |
| $T$   | $T \rightarrow F T_R$     | <i>synch</i>               |                           | $T \rightarrow F T_R$ | <i>synch</i>               | <i>synch</i>               |
| $T_R$ |                           | $T_R \rightarrow \epsilon$ | $T_R \rightarrow * F T_R$ |                       | $T_R \rightarrow \epsilon$ | $T_R \rightarrow \epsilon$ |
| $F$   | $F \rightarrow \text{id}$ | <i>synch</i>               | <i>synch</i>              | $F \rightarrow ( E )$ | <i>synch</i>               | <i>synch</i>               |

***synch***: the driver pops current nonterminal  $A$  and skips input till  
synch token or skips input until one of  $\text{FIRST}(A)$  is found

# Phrase-Level Recovery

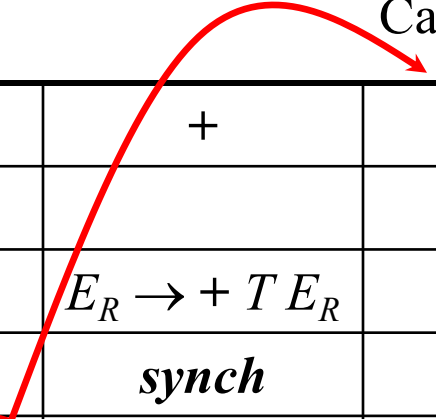
Change input stream by inserting missing tokens

For example: **id id** is changed into **id \* id**

Pro: Can be fully automated

Cons: Recovery not always intuitive

Can then continue here



|       | <b>id</b>                 | <b>+</b>                      | <b>*</b>                  | <b>(</b>              | <b>)</b>                      | <b>\$</b>                     |
|-------|---------------------------|-------------------------------|---------------------------|-----------------------|-------------------------------|-------------------------------|
| $E$   | $E \rightarrow T E_R$     |                               |                           | $E \rightarrow T E_R$ | <i>synch</i>                  | <i>synch</i>                  |
| $E_R$ |                           | $E_R \rightarrow + T E_R$     |                           |                       | $E_R \rightarrow \varepsilon$ | $E_R \rightarrow \varepsilon$ |
| $T$   | $T \rightarrow F T_R$     | <i>synch</i>                  |                           | $T \rightarrow F T_R$ | <i>synch</i>                  | <i>synch</i>                  |
| $T_R$ | <i>insert *</i>           | $T_R \rightarrow \varepsilon$ | $T_R \rightarrow * F T_R$ |                       | $T_R \rightarrow \varepsilon$ | $T_R \rightarrow \varepsilon$ |
| $F$   | $F \rightarrow \text{id}$ | <i>synch</i>                  | <i>synch</i>              | $F \rightarrow ( E )$ | <i>synch</i>                  | <i>synch</i>                  |

*insert \**: driver inserts missing \* and retries the production

# Error Productions

$$E \rightarrow T E_R$$

$$E_R \rightarrow + T E_R \mid \varepsilon$$

$$T \rightarrow F T_R$$

$$T_R \rightarrow * F T_R \mid \varepsilon$$

$$F \rightarrow ( E ) \mid \mathbf{id}$$

Add “*error production*”:

$$T_R \rightarrow F T_R$$

to ignore missing \*, e.g.: **id id**

Pro: Powerful recovery method

Cons: Manual addition of productions

|                      | <b>id</b>                   | <b>+</b>                      | <b>*</b>                  | <b>(</b>              | <b>)</b>                      | <b>\$</b>                     |
|----------------------|-----------------------------|-------------------------------|---------------------------|-----------------------|-------------------------------|-------------------------------|
| <i>E</i>             | $E \rightarrow T E_R$       |                               |                           | $E \rightarrow T E_R$ | <b><i>synch</i></b>           | <b><i>synch</i></b>           |
| <i>E<sub>R</sub></i> |                             | $E_R \rightarrow + T E_R$     |                           |                       | $E_R \rightarrow \varepsilon$ | $E_R \rightarrow \varepsilon$ |
| <i>T</i>             | $T \rightarrow F T_R$       | <b><i>synch</i></b>           |                           | $T \rightarrow F T_R$ | <b><i>synch</i></b>           | <b><i>synch</i></b>           |
| <i>T<sub>R</sub></i> | $T_R \rightarrow F T_R$     | $T_R \rightarrow \varepsilon$ | $T_R \rightarrow * F T_R$ |                       | $T_R \rightarrow \varepsilon$ | $T_R \rightarrow \varepsilon$ |
| <i>F</i>             | $F \rightarrow \mathbf{id}$ | <b><i>synch</i></b>           | <b><i>synch</i></b>       | $F \rightarrow ( E )$ | <b><i>synch</i></b>           | <b><i>synch</i></b>           |

# Conclusions...